

Informe de la pràctica 6 Busos de comunicació II

Introducció

Aquesta sisena pràctica suposa l'evolució natural en l'estudi de les comunicacions digitals en sistemes encastats. Després de dominar el bus I2C a la pràctica anterior, l'objectiu ara és comprendre i implementar el bus **SPI (Serial Peripheral Interface)**. Aquest protocol, dissenyat per a distàncies curtes però a velocitats molt superiors, ens permetrà interactuar amb components que requereixen moure grans volums de dades o respostes gairebé instantànies, com les targetes de memòria SD o els lectors de radiofreqüència (RFID).

1.1. Objectius

La finalitat d'aquest projecte és entendre les particularitats físiques i de programari del protocol SPI, aplicant-les a casos d'ús professionals. Els objectius específics són els següents:

- **Assimilar l'arquitectura del bus SPI:** Comprendre la topologia Mestre-Esclau, la transmissió de dades síncrona i bidireccional (*Full Duplex*), i el rol fonamental de la línia de selecció *Slave Select* (SS).
- **Gestió de Sistemes de Fitxers:** Configurar l'ESP32 per inicialitzar una targeta memòria física (micro SD) i llegir el contingut d'arxius de text plans utilitzant les llibreries natives del nucli d'Arduino.
- **Lectura de Sensòrica de Seguretat:** Establir comunicació amb un mòdul RFID-RC522 per detectar l'apropament d'etiquetes intel·ligents i extreure'n l'Identificador Únic (UID) mitjançant ràfegues d'espera activa (*polling*).
- **Resolució de Conflictes de Maquinari (Multiesclau):** Demostrar la capacitat de connectar i gestionar diversos dispositius SPI (SD i RFID) sobre les mateixes línies físiques sense causar col·lisions de dades.
- **Desenvolupament d'un Sistema IoT (Projecte Final):** Integrar la lectura RFID, el registre d'accessos en format *datalogger* (desant la data i l'hora exacta a la SD) i un Servidor Web autònom per monitorar les dades des de qualsevol dispositiu connectat a la xarxa local Wi-Fi.

1.2. Material Utilitzat

Per desenvolupar aquesta sèrie d'exercicis s'ha fet servir el següent maquinari i programari, reflectint la complexitat creixent del muntatge:

Maquinari (Hardware):

- **Microcontrolador:** Placa de desenvolupament ESP32-S3 (actuant com a Mestre del bus SPI).
- **Esclau 1:** Mòdul adaptador de targetes micro SD amb una targeta de memòria.

- **Esclau 2:** Mòdul lector RFID-RC522 (radiofreqüència).
- **Accessoris:** Targetes i clauers RFID (com el típic clauer blau de 13.56 MHz) per realitzar les lectures.
- Placa de proves (*proto-board*) i cables de connexió Dupont per estructurar les línies compartides.

Programari (Software):

- Entorn Visual Studio Code amb l'extensió PlatformIO.
- Llibreries de sistema natives: [SPI.h](#), [SD.h](#), [WiFi.h](#), [WebServer.h](#) i [time.h](#).
- Llibreries de tercers: [MFRC522.h](#) (de Miguel Balboa) per a la decodificació del mòdul RFID.

1.3. Conceptes Claus: El Protocol SPI

Per justificar les decisions preses al codi i entendre el muntatge físic (visible a les fotografies de l'informe), és imprescindible repassar les característiques tècniques del bus SPI, desenvolupat originàriament per Motorola l'any 1980:

- **Comunicació Full Duplex i Alta Velocitat:** A diferència de l'I2C, l'SPI utilitza línies independents per enviar i rebre dades, permetent al Mestre i a l'Esclau parlar exactament al mateix temps (*Full Duplex*). Com que manca d'un protocol pesat de confirmació (ACK), assolix velocitats de transmissió extremadament ràpides (fins a 80 MHz a l'ESP32).
- **Línies Compartides:** El bus requereix un mínim de 3 línies base que connecten tots els dispositius en paral·lel:
 - **SCK (Clock):** Senyal de rellotge generada sempre pel Mestre per sincronitzar la transmissió.
 - **MOSI (Master-Out, Slave-In):** Línia per on el Mestre envia dades cap als Esclaus.
 - **MISO (Master-In, Slave-Out):** Línia per on l'Esclau respon al Mestre.
- **La Línia Exclusiva SS (Slave Select):** Atès que l'SPI no utilitza un sistema d'adreces de programari com l'I2C, necessita afegir un quart cable **físic i exclusiu** per a cada dispositiu esclau connectat. Per defecte, l'ESP32 manté totes les línies SS en estat ALT (**HIGH**); quan vol establir comunicació amb un esclau concret, posa la seva línia SS específica a BAIX (**LOW**), despertant només aquell xip i ignorant la resta.

Desenvolupament i Implementació: Exercici Pràctic 1 (Lectura de Memòria SD)

En aquesta pràctica fem el salt al bus **SPI (Serial Peripheral Interface)**. A diferència de l'I2C, que utilitza adreces per identificar els dispositius, l'SPI utilitza línies físiques dedicades per a cada esclau, conegudes com a **SS (Slave Select)** o CS (Chip Select). Aquest primer exercici consisteix a establir comunicació amb un mòdul lector de targetes micro SD per extreure i llegir el text d'un arxiu **.txt** prèviament desat al seu interior.

1. Configuració de l'Entorn (**platformio.ini**)

Aquest cop, el fitxer de configuració **.ini** és extremadament minimalista:

- **Absència de lib_deps:** No hem hagut d'afegir cap llibreria externa descarregable. Això es deu al fet que les llibreries per gestionar el maquinari SPI i els sistemes de fitxers SD (**SPI.h** i **SD.h**) ja venen integrades de forma nativa al nucli del framework d'Arduino per a l'ESP32.
- **monitor_speed = 9600:** Cal destacar que la velocitat del terminal sèrie s'ha ajustat a 9600 bauds per coincidir exactament amb el **Serial.begin(9600)** del codi font, evitant així l'aparició de caràcters estranys a la pantalla.

2. Codi Principal: Lectura d'Arxius per SPI (**main.cpp**)

El codi s'executa de manera seqüencial dins de la funció **setup()**, llegint l'arxiu sencer d'una sola tirada i deixant el **loop()** completament buit.

Inicialització del Bus i el Dispositiu:

- **#include <SPI.h>** i **#include <SD.h>:** Importen les funcions necessàries per parlar amb el bus (línies MOSI, MISO i SCK) i per interpretar el sistema de fitxers de la targeta (FAT16/FAT32).
- **SD.begin(4):** Aquesta és la línia clau de la configuració SPI. Inicialitza el bus indicant-li al microcontrolador ESP32 que el pin digital encarregat d'actuar com a **Slave Select (SS)** és el **GPIO 4**. Quan aquesta instrucció s'executa, l'ESP32 posa el pin 4 a nivell baix (LOW) per "despertar" la targeta SD i avisar-la que vol parlar amb ella. Si no hi ha cap targeta connectada, o el cable SS falla, la funció retorna **false** i el programa s'atura imprimint un error.

Obertura i Lectura del Fitxer:

- **myFile = SD.open("archivo.txt");**: Un cop la targeta SD està activa al bus SPI, se li envia l'ordre de buscar i obrir el fitxer anomenat "archivo.txt" en mode de només lectura.
- **while (myFile.available());**: S'inicia un bucle que comprova si queden bytes per llegir dins del fitxer.
- **Serial.write(myFile.read());**: Aquesta és l'acció principal. La instrucció **myFile.read()** sol·licita el següent byte (caràcter) a la targeta SD. La targeta l'envia per la línia de dades **MISO (Master-In, Slave-Out)** cap a l'ESP32. Immediatament, l'ESP32 agafa aquest caràcter i l'imprimeix pel terminal de l'ordinador mitjançant **Serial.write**.
- **myFile.close();**: Un cop s'ha acabat de llegir l'arxiu, és vital tancar-lo per alliberar la memòria de l'ESP32 i donar per finalitzada la transacció amb la targeta.

Sortida pel Terminal de Control

En executar el programa, el microcontrolador actua com un pont: llegeix el text guardat físicament a la memòria SD i ens l'ensenya per la pantalla del nostre ordinador.

Si suposem que dins de la targeta hi hem desat prèviament un fitxer de text amb la frase *"Hola, això es una prova per a la Practica 6"*, la sortida del monitor sèrie seria la següent:

Plaintext

Iniciando SD ...
inicializacion exitosa
archivo.txt:
Hola, això es una prova per a la Practica 6

En cas de no haver introduït la targeta SD al mòdul, o si els cables del bus SPI (MOSI, MISO, SCK o SS) estiguessin mal connectats, el programa executaria l'estructura de seguretat inicial i mostraria:

Plaintext
Iniciando SD ...
No se pudo inicializar

Anàlisi del Temps Lliure del Processador (Lectura Targeta SD per SPI)

Aquest primer exercici de la Pràctica 6 presenta un perfil de càrrega de processament extremadament asimètric, on tota la feina es concentra exclusivament durant els primers instants de vida del programa.

1. Càrrega de treball inicial i gestió del bus (setup): En engegar la placa, el processador dedica tots els seus esforços a inicialitzar el bus SPI i muntar el sistema de fitxers de la targeta SD. Durant el bucle de lectura `while (myFile.available())`, la CPU està plenament ocupada fent de pont: demana un byte a la targeta a través de la línia MISO i l'envia gairebé a l'instant cap a l'ordinador pel port sèrie.

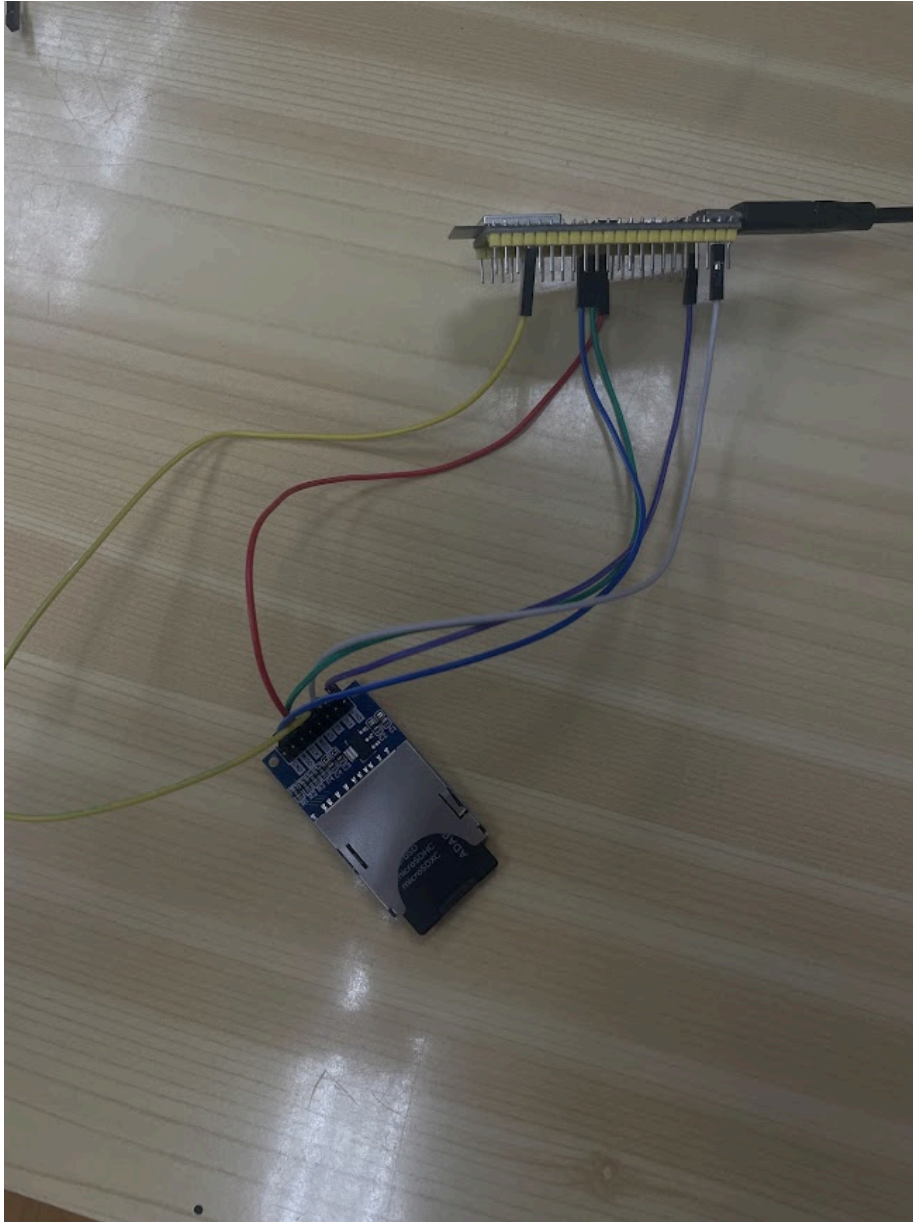
Nota de rendiment: Llegir un fitxer byte a byte no és la tècnica més optimitzada des del punt de vista del programari (seria més eficient llegir blocs sencers de dades). No obstant això, gràcies a l'alta velocitat del maquinari SPI, tota l'operació d'extreure el text i imprimir-lo es completa en tan sols uns quants mil·lisegons.

2. Inactivitat total i absoluta (loop): La característica més destacable d'aquest codi és què passa quan s'arriba al final del text. Després de la instrucció `myFile.close()`, el programa cau dins de la funció `loop()`, la qual es troba totalment buida.

En un microcontrolador tradicional de 8 bits sense sistema operatiu, un `loop()` buit faria que el processador girés sobre si mateix executant salts buits i consumint energia innecessàriament al 100% de la seva capacitat. Tanmateix, com que l'ESP32 treballa sota la capa del sistema operatiu **FreeRTOS**, el comportament és molt més eficient. En detectar que la tasca principal no conté instruccions, el planificador (*scheduler*) delega el control immediatament a la **Tasca Inactiva (Idle Task)**.

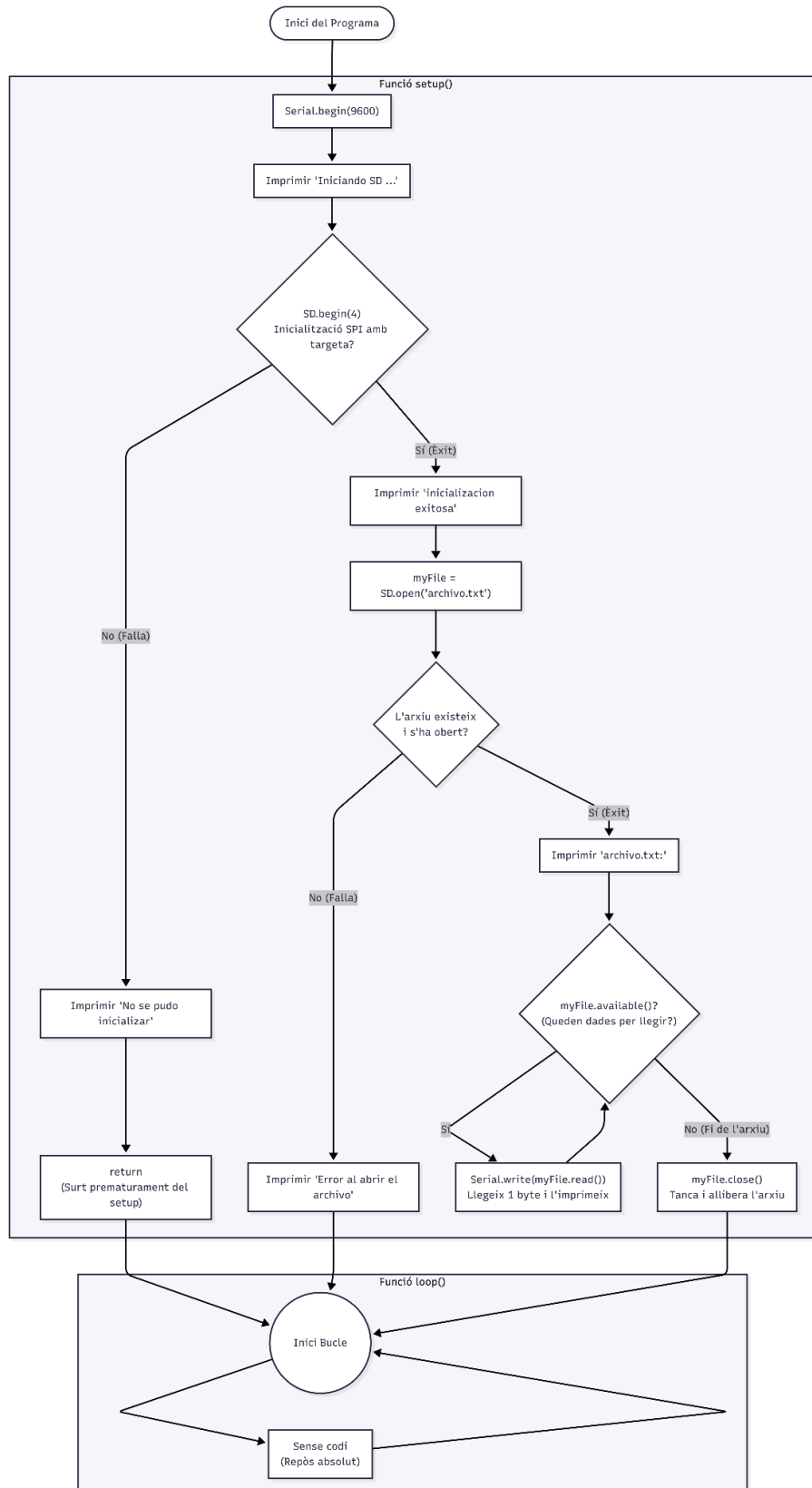
Conclusió: El rendiment d'aquest programa es resumeix en un pic de processament molt curt (una fracció de segon) durant l'arrencada, seguit d'una caiguda a un **temps lliure**

proper al 100% de forma indefinida. Un cop llegit l'arxiu, tant el processador com les línies del bus SPI (MOSI, MISO i SCK) queden completament silenciades i a l'espera.



Diagrames de flux i temps

Diagrama de flux

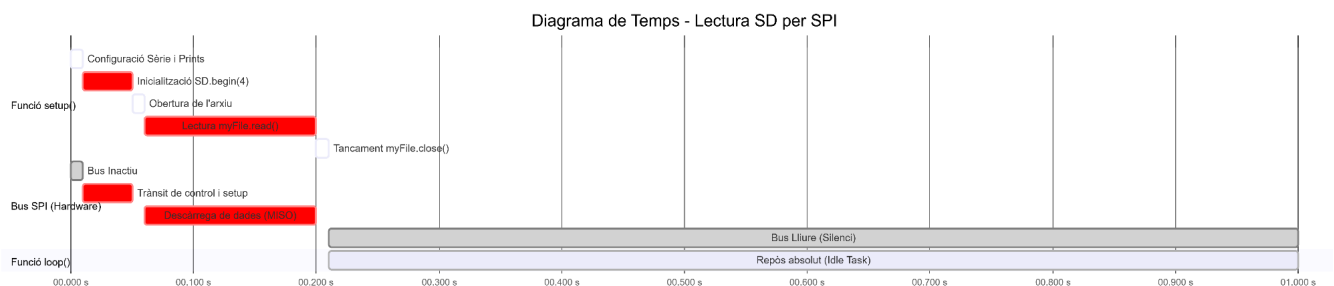


Doble barrera de seguretat (Rombes): S'hi observa clarament com l'estructura de la funció `setup()` depèn de dues grans condicions. Primer es comprova que el maquinari estigui operatiu (`SD.begin(4)`) i després es valida que l'arxiu de text estigui íntegre (`if(myFile)`). Si qualsevol de les dues falla, el programa salta a la fi i no intenta fer res més.

El bucle d'extracció: A la part central, el bloc `myFile.available()` genera un sub-bucle que només gira mentre quedin lletres a la memòria. Mostra visualment com la lectura es fa pas a pas abans de tancar el fitxer.

Mort prematura vs. cicle de vida: Un detall important del diagrama és que, tant si hi ha hagut un error (via l'ordre `return`) com si l'arxiu s'ha llegit amb èxit, tots els camins acaben derivant cap al mateix punt final: un `loop()` buit que mantindrà el sistema en suspensió de forma permanent.

Diagrama de temps



La negociació inicial (10 - 50 ms): En cridar la funció `SD.begin(4)`, l'ESP32 activa el pin CS/SS i comença a intercanviar polsos de rellotge i comandaments de control pel bus SPI (línies MOSI i MISO) per muntar el sistema de fitxers. És un procés crític (franja vermella) on el maquinari treballa a fons.

L'extracció de dades (60 - 200 ms): Durant el bucle `while`, el bus SPI torna a patir una càrrega sostinguda. La línia MISO (Master-In, Slave-Out) està constantment ocupada transferint els caràcters del text des de la memòria SD cap a la memòria de l'ESP32. Aquesta franja serà més o menys llarga depenent de la quantitat de text que contingui l'arxiu.

L'aturada definitiva (A partir dels 210 ms): Immediatament després d'executar `myFile.close()`, la transacció es dona per acabada. Tal com mostra la secció inferior, el codi entra al `loop()` buit, el bus SPI queda totalment mut (silenci) i el processador cau a la Tasca Inactiva de l'RTOS, consumint el mínim d'energia possible fins que es reiniciï la placa.

Desenvolupament i Implementació: Exercici Pràctic 2 (Lectura d'Etiqueta RFID)

En aquest segon exercici continuem utilitzant el bus d'alta velocitat **SPI**, però aquest cop per establir comunicació amb un mòdul lector de targetes RFID basat en el xip **RC522**.

L'objectiu del codi és detectar la presència d'una etiqueta intel·ligent al camp magnètic de l'antena, comunicar-s'hi i extreure'n el seu Identificador Únic (UID).

1. Configuració de l'Entorn (**platformio.ini**)

Per poder interactuar amb els registres complexos del xip RC522 sense haver de programar el protocol de baix nivell des de zero, el fitxer **.ini** inclou una adaptació clau:

- **lib_deps = miguelbalboa/MFRC522@^1.4.11**: Aquesta instrucció ordena a PlatformIO que descarregui i enllaci la llibreria oficial més estandarditzada per a aquests mòduls. Gràcies a aquesta llibreria, l'ESP32 podrà traduir funcions senzilles d'alt nivell en trames de dades SPI perfectament sincronitzades cap a l'esclau.
- Es manté la velocitat del monitor sèrie a **9600** bauds per coherència amb el codi font.

2. Codi Principal: Lectura del UID (**main.cpp**)

L'arquitectura d'aquest programa estableix un bucle d'escolta contínua dissenyat per no bloquejar el processador mentre s'espera l'apropament d'una targeta.

Definició de Pins i Objectes:

- **#define SS_PIN 10 i #define RST_PIN 9**: A diferència del bus I2C, a l'SPI necessitem un pin de selecció físic. El pin 10 (SS) és la línia *Slave Select* que l'ESP32 posarà a nivell baix per avisar al lector RFID que vol comunicar-s'hi. El pin 9 s'utilitza per fer un reinici (Reset) físic del mòdul.
- **MFRC522 mfrc522(SS_PIN, RST_PIN);**: Instància l'objecte lògic que controlarà el mòdul, vinculant-lo als pins definits anteriorment.

Inicialització (**setup**):

- **SPI.begin();**: Desperta i configura el maquinari del bus SPI integrat a l'ESP32 (assignant automàticament els pins SCK, MOSI i MISO per defecte).
- **mfrc522.PCD_Init();**: Aquesta és una funció específica de la llibreria. *PCD* fa referència a *Proximity Coupling Device* (el lector). Inicialitza els registres del RC522 i activa l'antena emissora perquè comenci a generar el camp electromagnètic.

Detecció i Lectura (loop): El bucle principal repeteix un procés de filtratge estricte a gran velocitat:

- **if (mfrc522.PICC_IsNewCardPresent())**: Actua com la primera barrera. La placa pregunta a l'esclau SPI si hi ha una targeta (*PICC*) nova al camp d'acció. Si no n'hi ha cap, la funció retorna **false** i el programa no fa res, evitant operacions innecessàries.
- **if (mfrc522.PICC_ReadCardSerial())**: Si hi ha una targeta present, aquesta funció intenta seleccionar-la i llegir-ne les dades.
- **El bucle for d'impressió**: Com que el UID de la targeta (l'identificador únic) es compon normalment de 4 bytes, el codi utilitza un bucle per iterar sobre **mfrc522.uid.size**. A cada iteració, agafa un byte (**uidByte[i]**) i l'imprimeix al terminal en format hexadecimal (**HEX**).

- A més, inclou una petita condició de formatatge gràfic (`mfr522.uid.uidByte[i] < 0x10 ? " 0" : " "`) que afegeix un zero al davant si el valor hexadecimal és menor de 10, garantint que tots els bytes s'imprimeixin sempre amb dues xifres (ex: `0A` en comptes d'`A`).
- `mfr522.PICC_HaltA();`: Aquesta és una instrucció de control **vital**. Quan acaba d'imprimir el codi, envia l'ordre de "pausa" (*Halt*) a la targeta llegida. Això evita que l'ESP32 llegeixi exactament la mateixa targeta milers de vegades per segon mentre l'usuari la mantingui recolzada sobre l'antena.

Sortida pel Terminal de Control

Un cop carregat el codi, el sistema inicialitza el lector i queda a l'espera. Cada cop que apropem una targeta o un clauer RFID (com els blaus típics dels kits), l'ESP32 llegeix les dades per SPI i les representa a la pantalla.

La sortida per la consola mostra el text inicial i, tot seguit, els UID de les diferents etiquetes que s'han anat apropant:

Plaintext

Lectura del UID

Card UID: A3 4F 8B 12

Card UID: 55 B1 2C 09

Card UID: A3 4F 8B 12

Anàlisi del Temps Lliure del Processador (Lectura RFID per SPI)

A diferència dels exercicis anteriors on el processador queia en un repòs absolut gràcies a l'ús de `delay()` o al disseny basat en esdeveniments, l'anàlisi de rendiment d'aquest codi RFID ens mostra un escenari diametralment oposat: **l'espera activa (Polling)**.

1. Saturació del Bus i la CPU per Polling: Si observem la funció `loop()`, veurem que no hi ha cap instrucció de pausa o bloqueig. El microcontrolador executa contínuament la instrucció `mfr522.PICC_IsNewCardPresent()` a la màxima velocitat possible.

- Això significa que l'ESP32 està constantment obrint la línia SS (Slave Select), enviant senyals de rellotge (SCK) i preguntant per la línia MOSI: *"Hi ha alguna targeta ara? I ara? I ara?"*.
- Com a resultat, el bus SPI presenta un trànsit de control ininterromput i el nucli del processador encarregat d'executar aquest bucle treballa a ple rendiment, impedit que el sistema operatiu FreeRTOS derivi el control a la Tasca Inactiva (*Idle Task*).

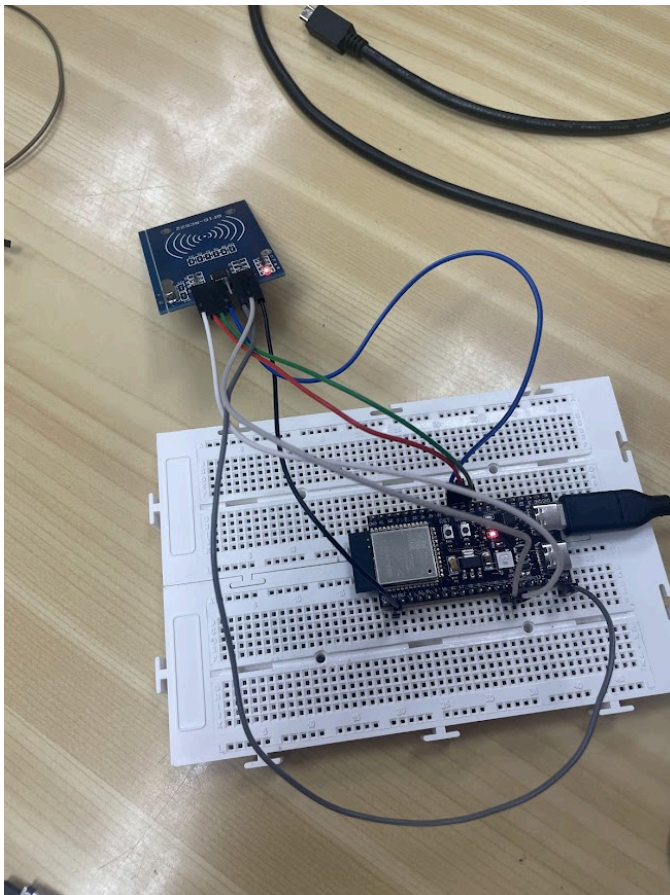
2. El pic de lectura útil: Tota aquesta enorme càrrega de treball només dona fruit en el moment exacte en què l'usuari apropa una etiqueta al camp magnètic. En aquell instant precís, el programa entra a la condició, executa `PICC_ReadCardSerial()`, transfereix els

4 bytes del UID pel bus SPI i els imprimeix pel terminal sèrie. Aquesta operació "útil" dura prou feines uns quants mil·lisegons, per tornar immediatament a l'esgotador bucle de preguntes constants.

3. Àrees d'optimització (Cap al model IoT professional): Aquest codi és perfecte i molt fiable com a exemple educatiu per entendre el protocol SPI, però el seu temps lliure és pràcticament nul. En un dispositiu IoT real alimentat per bateries, aquest disseny esgotaria l'energia ràpidament. Per alliberar la CPU i aconseguir el 99% de temps lliure que vèiem en pràctiques passades, s'haurien d'aplicar dues solucions:

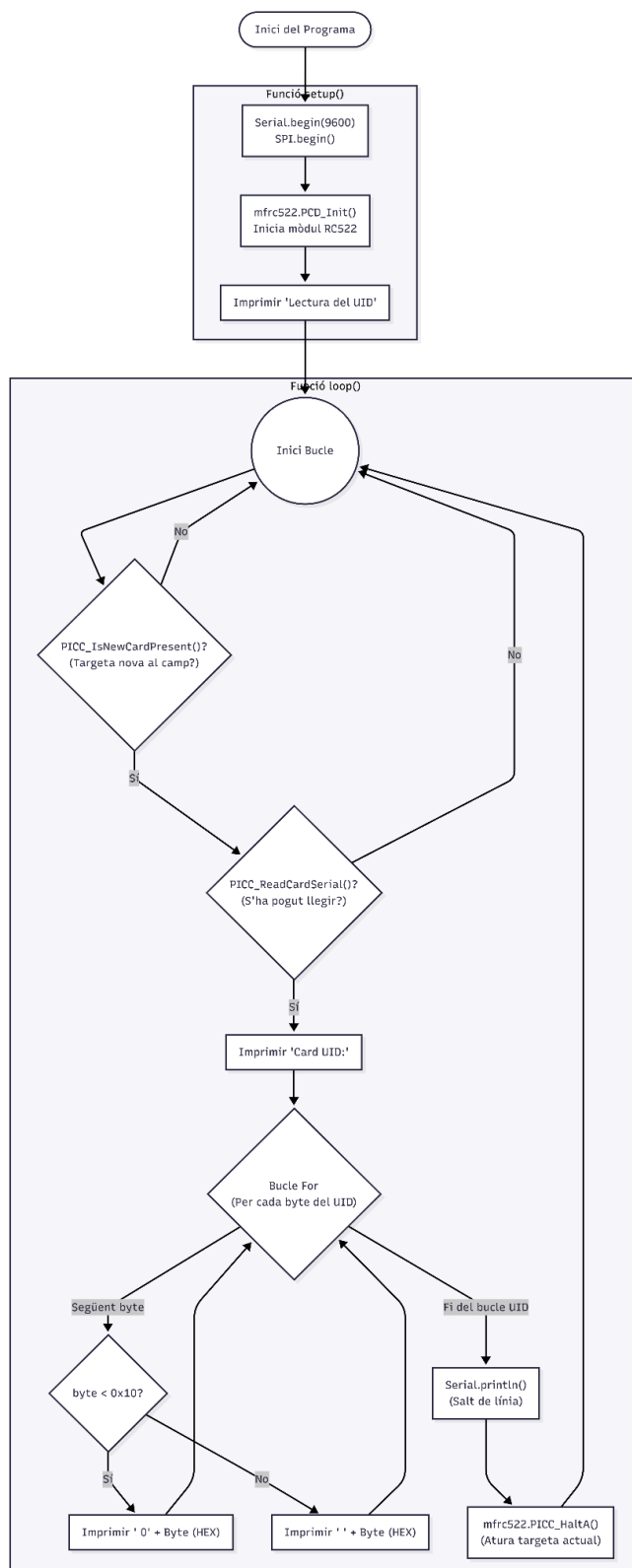
- **Solució bàsica:** Afegir un petit `delay(50)` al final del bucle. Això rebaixaria la freqüència de les preguntes a 20 vegades per segon, cedint el control a l'RTOS entre lectura i lectura sense que l'usuari notés cap retard.
- **Solució avançada (Interrupcions):** El xip RC522 disposa d'un pin físic anomenat `IRQ` (Interrupt Request). En lloc de preguntar constantment pel bus SPI, es podria programar l'ESP32 per adormir-se completament fins que el lector detectés magnèticament una targeta i enviés un pols elèctric per despertar el processador.

Conclusió: En aquesta implementació concreta, el processador dedica **gairebé el 100% del seu temps a estar ocupat**, demostrant que una comunicació d'alta velocitat com l'SPI no garanteix per si sola l'eficiència energètica si el programari obliga el maquinari a treballar sense descans. L'exercici ens ensenya que per a dispositius de monitorització contínua, és vital plantejar estratègies que evitin l'espera activa.



Diagrames de flux i temps

Diagrama de flux



El Filtre d'Espera Activa (Rombes Principals):

El diagrama il·lustra perfectament el procés de *polling*. Les dues primeres condicions

(`IsNewCardPresent` i

`ReadCardSerial`) actuen com a

vàlvules. Si la targeta no hi és o no es pot llegir amb claredat, el flux és rebutjat cap a l'inici del bucle de forma immediata, representant gràficament l'alta freqüència de gir d'aquest codi quan no hi ha cap etiqueta a prop.

L'Extracció de Dades (Bucle For): Un cop superades les barreres, s'entra a un sub-bucle que itera en funció de la mida de l'identificador

(`mfrc522.uid.size`). S'hi veu

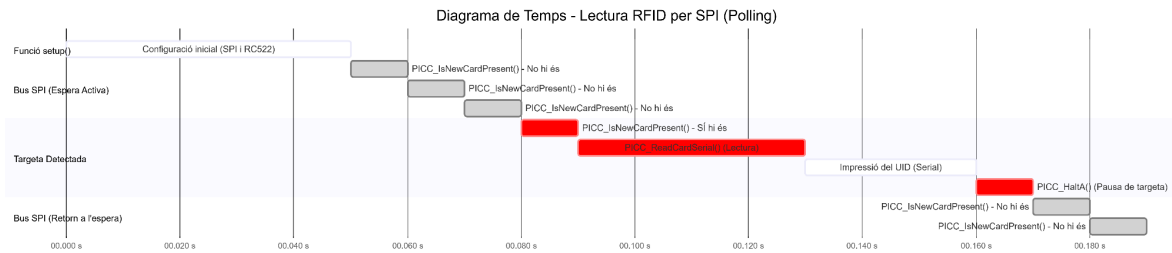
clarament el pas de formatatge on el codi decideix si cal imprimir un zero extra al davant de cada nombre hexadecimal perquè quedi alineat.

El Fre de Mà (HaltA): La darrera caixa abans de reiniciar el bucle mostra el pas de finalització

(`mfrc522.PICC_HaltA()`).

Visualment es demostra que, sense aquesta caixa, el diagrama tornaria a entrar a llegir exactament la mateixa targeta al següent cicle de forma infinita.

Diagrama de temps



L'Espera Activa (*Polling* sostingut): Un cop finalitzat el `setup()`, el diagrama mostra una seqüència repetitiva (franges grises) on l'ESP32 pregunta contínuament al lector si hi ha una targeta mitjançant la instrucció `PICC_IsNewCardPresent()`. Aquesta interrogació s'executa desenes o centenars de vegades per segon, mantenint el bus SPI i la CPU completament ocupats sense realitzar cap tasca productiva.

El Pic de Transacció (Detecció): En el moment hipotètic (als 80 ms del gràfic) en què s'introdueix una targeta magnètica, el flux canvia. S'encadenen tres accions crítiques de forma gairebé instantània: la lectura efectiva dels registres (`PICC_ReadCardSerial()`), la impressió de les dades hexadecimal cap al terminal sèrie i la instrucció de fre (`PICC_HaltA()`) per posar l'etiqueta a dormir.

El Bucle Infinit: Immediatament després d'adormir la targeta acabada de llegir, s'observa com el programa no descansa pas, sinó que reprèn automàticament l'esgotador procés de preguntar `PICC_IsNewCardPresent()` una vegada i una altra a l'espera d'un nou esdeveniment.

Desenvolupament i Implementació: Exercici de Pujada de Nota (SD + RFID + Servidor Web)

Aquest darrer apartat aborda el repte final plantejat a l'enunciat: la integració de dos perifèrics SPI compartint el mateix bus (lector RFID i memòria SD), l'enregistrament de dades (*data logging*) en un fitxer `.log` amb segell de temps, i la creació d'un servidor web asíncron per monitorar les lectures en temps real.

1. Configuració de l'Entorn (`platformio.ini`)

El fitxer de configuració reflecteix les necessitats d'aquest sistema IoT:

- `lib_deps = miguelbalboa/MFRC522@^1.4.11`: Només cal descarregar externament la llibreria per al control del xip RFID. Tota la resta de funcions complexes (connexió Wi-Fi, Servidor Web HTTP, sincronització de temps NTP i gestió del sistema de fitxers de la targeta SD) s'han resolt utilitzant les llibreries natives integrades al nucli d'Arduino per a l'ESP32, fet que optimitza l'espai i evita conflictes de dependències.

- **monitor_speed = 115200**: S'ha elevat la velocitat del port sèrie a 115200 bauds respecte als exercicis anteriors, una pràctica molt recomanable quan s'utilitzen connexions Wi-Fi, ja que el sistema emet missatges de depuració a gran velocitat.

2. Codi Principal: Resolució de Maquinari i Lògica Multitasca (**main.cpp**)

El codi font està dissenyat per gestionar simultàniament comunicacions de maquinari i de xarxa. L'aspecte més crític d'aquest programa és com es resol el conflicte del bus SPI per a dos perifèrics.

Resolució del Maquinari (Compartir el Bus SPI): L'enunciat demana explícitament descriure com es resol la connexió de dos esclaus. El bus SPI permet connectar múltiples dispositius en paral·lel compartint exclusivament tres cables: Relotge (**SCK_PIN 36**), Sortida del Mestre (**MOSI_PIN 35**) i Entrada del Mestre (**MISO_PIN 37**). Per evitar que el lector RFID i la targeta SD "parlin" alhora i corrompin les dades, s'ha assignat un cable independent per a cadascun anomenat **CS (Chip Select)** o **SS (Slave Select)**:

- **#define RFID_CS_PIN 10**
- **#define SD_CS_PIN 4** El microcontrolador mantindrà els dos pins a nivell ALT (inactius) per defecte. Quan vulgui llegir una targeta, posarà el pin 10 a nivell BAIX; quan vulgui escriure al fitxer, pujarà el 10 i baixarà el pin 4. Aquesta gestió es fa automàticament gràcies a les llibreries instanciades.

Inicialització i Sincronització (**setup**):

- **SPI.begin(SCK_PIN, MISO_PIN, MOSI_PIN);**: Configura explícitament els pins que l'ESP32-S3 utilitzarà per a les línies principals del bus.
- **SD.begin(SD_CS_PIN, SPI)**: Aquesta instrucció és fonamental. A més d'indicar-li quin és el seu pin CS (4), se li passa per referència l'objecte **SPI** global. Això força a la llibreria de la SD a utilitzar el bus compartit en lloc d'intentar obrir-ne un de nou, evitant errors de maquinari.
- **configTime(gmtOffset_sec, daylightOffset_sec, ntpServer);**: Com que l'ESP32 no té un rellotge RTC físic com el de la pràctica anterior, utilitza la connexió Wi-Fi per demanar l'hora exacta a un servidor d'internet (**pool.ntp.org**) i ajusta l'hora espanyola (+3600 segons).

El Bucle Principal i l'Enregistrament (**loop**):

- **server.handleClient();**: Escolta constantment si algun navegador demana accedir a la pàgina web. La pàgina (**handleRoot**) inclou una etiqueta **<meta http-equiv='refresh' content='2'>** perquè el mòbil de l'usuari es recarregui automàticament cada 2 segons i mostri la darrera targeta detectada.
- **Lectura i Formatatge**: Quan es detecta una targeta amb **PICC_IsNewCardPresent()**, el codi extrau el UID i el converteix en un **String** hexadecimal net (**codiHex.toUpperCase()**) per fer-lo fàcilment llegible.

- **Datalogging (Escriptura a la SD):** S'obté l'hora sincronitzada (`getLocalTime`) i es formatarà en una cadena de text (ex: `11/05/2026 18:30:00`). Tot seguit, s'obre el fitxer amb `SD.open("/fichero.log", FILE_APPEND)`. L'ús de `FILE_APPEND` és crucial aquí: en comptes de sobreesciure i esborrar el fitxer, afegeix la nova línia de text al final de les dades existents, creant un autèntic registre històric d'accessos. Finalment, es tanca el fitxer (`dataFile.close()`) per assegurar que les dades es desen físicament a la memòria.

Sortida pel Terminal de Control

En engegar la placa, el monitor sèrie mostra el procés d'arrencada de tots els subsistemes (SPI, RFID, SD i Wi-Fi). Un cop operatiu, registra cada nova lectura.

La sortida observable és la següent:

```
Plaintext
Lector RFID inicialitzat.
Targeta SD inicialitzada correctament.
Connectant a la xarxa: Nautilus
.....
Connexió establerta!
Adreça IP del servidor: 192.168.1.50
Sincronitzant l'hora amb el servidor NTP...
Lectura detectada: A3 4F 8B 12
Registre guardat correctament a fichero.log
Lectura detectada: 55 B1 2C 09
Registre guardat correctament a fichero.log
```

Si en aquest moment extraguéssim la targeta micro SD de la placa i la connectem a l'ordinador, hi trobaríem un arxiu anomenat `fichero.log` amb el següent contingut intern:

```
Plaintext
[11/05/2026 18:30:15] UID detectat: A3 4F 8B 12
[11/05/2026 18:35:42] UID detectat: 55 B1 2C 09
```

Anàlisi del Temps Lliure del Processador (IoT SPI: SD + RFID + Servidor Web)

Aquest darrer projecte ens presenta l'escenari més complex de tota la pràctica: un microcontrolador que ha de fer malabars entre atendre peticions de xarxa, controlar el sistema de fitxers d'una memòria física i interrogar constantment un sensor magnètic. Tot això té un impacte directe en el temps lliure de la CPU.

1. El Polling Combinat (Doble càrrega contínua): En observar el bucle principal, no hi trobem cap instrucció de repòs general. El processador està sotmès a una **espera activa (polling) doble**:

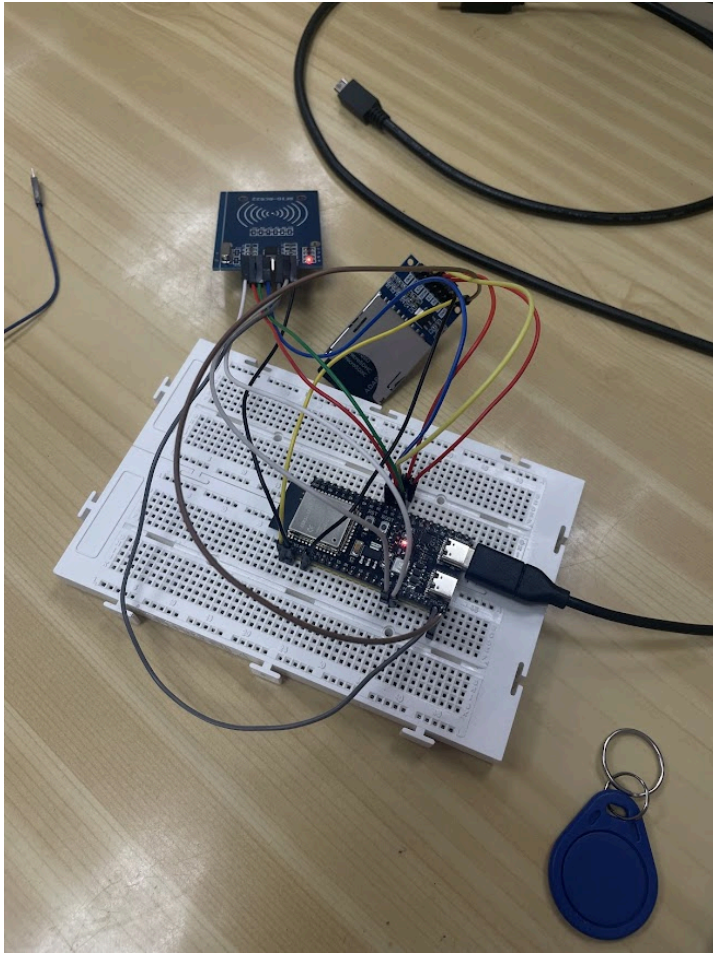
- Per una banda, executa `server.handleClient()` a cada cicle per revisar si hi ha trànsit a la targeta Wi-Fi.
- Immediatament després, llança la petició SPI `mfr522.PICC_IsNewCardPresent()` per comprovar l'estat de l'antena RFID. Com a conseqüència, el nucli del processador treballa a un ritme frenètic i la línia SPI no descansa mai, ja que l'ESP32 no para de fer baixar i pujar el pin `RFID_CS_PIN` (10) per interrogar el lector. El temps lliure efectiu cedit a la Tasca Inactiva (Idle Task) és pràcticament nul.

2. El Coll d'Ampolla de l'Escriptura (Targeta SD): L'únic moment on el flux de *polling* s'atura és quan es detecta una targeta. En aquest instant, el processador canvia de context i ataca la targeta SD.

- L'operació d'obrir un fitxer, afegir-hi text al final (`FILE_APPEND`) i tancar-lo (`dataFile.close()`) és, computacionalment parlant, una de les operacions més lentes que pot fer un microcontrolador.
- Requereix moure dades pel bus SPI, interactuar amb el sistema d'arxius (FAT32) i esperar que la memòria flaix de la targeta es cremi físicament. Tot i que per a nosaltres són uns pocs mil·lisegons, per al processador és una eternitat on queda totalment bloquejat.

3. El Descans Forçat i la Sincronització (`delay(500)`): Paradoxalment, l'únic moment de la funció `loop()` on l'ESP32 pot descansar és just després de treballar de valent. Un cop s'ha desat la lectura a l'arxiu `.log` i s'ha aturat la targeta RFID, el codi executa un `delay(500)`. Aquest mig segon de bloqueig té dues funcions clau: dóna temps a l'usuari a retirar la targeta perquè no es llegeixi dues vegades seguides i, gràcies a FreeRTOS, adorm el processador enviant-lo a la Tasca Inactiva per alliberar la memòria cau i netejar els processos de xarxa pendents.

Conclusió: Ens trobem davant d'un sistema dissenyat per a la **màxima responsivitat** però amb una **baixa eficiència energètica**. El processador està retingut a prop del 100% de la seva capacitat fent guàrdia ("Hi ha una petició web? Hi ha una targeta RFID?"). Si aquest projecte fos un pany intel·ligent alimentat per bateries, aquest disseny les esgotaria en poques hores. No obstant això, per a un dispositiu connectat a la xarxa elèctrica (com un control d'accés fix en una oficina), aquest disseny garanteix que el sistema mai perdrà cap lectura ni deixarà d'actualitzar la pàgina web instantàniament.



Desenvolupament i Implementació: Exercici de Pujada de Nota (SD + RFID + Web)

Aquest darrer apartat aborda el repte final plantejat a l'enunciat: la integració de dos perifèrics esclaus compartint un mateix bus SPI (un lector RFID i una memòria SD), la creació d'un registre d'accessos en un fitxer `.log` amb l'hora exacta, i la visualització de la darrera targeta detectada mitjançant una pàgina web.

1. Configuració de l'Entorn (`platformio.ini`)

El fitxer de configuració es manté molt net i optimitzat per a un sistema de xarxa:

- `lib_deps = miguelbalboa/MFRC522@^1.4.11`: Novament, l'única dependència externa que necessitem instal·lar és la del lector RFID. Les llibreries per a la SD, el Wi-Fi, el servidor web i la gestió del temps (`time.h`) ja estan integrades al framework de l'ESP32.
- `monitor_speed = 115200`: Atès que treballarem amb connexions Wi-Fi i un servidor NTP, la velocitat del port sèrie s'ha elevat a 115200 bauds per processar els missatges de depuració a gran velocitat.

2. Codi Principal: Resolució de Maquinari Multiesclau i Datalogging (**main.cpp**)

La complexitat d'aquest codi recau a coordinar l'accés a les tres línies compartides del bus SPI (SCK, MOSI i MISO) per part de dos dispositius diferents sense que les dades es corrompin.

Resolució del conflicte SPI (Gestió dels CS): El protocol SPI estableix que cada esclau ha de tenir una línia independent anomenada *Chip Select* (CS) o *Slave Select* (SS). Quan aquesta línia està a nivell BAIX (LOW), l'esclau escolta; quan està a nivell ALT (HIGH), l'esclau ignora el bus. El gran problema sorgeix a l'arrencada: si la SD i l'RFID intenten inicialitzar-se a la vegada, col·lisionen. El codi ho resol magistralment al principi del **setup()**:

```
C++
pinMode(RFID_CS_PIN, OUTPUT);
digitalWrite(RFID_CS_PIN, HIGH);
pinMode(SD_CS_PIN, OUTPUT);
digitalWrite(SD_CS_PIN, HIGH);
```

Aquestes línies forcen manualment els pins CS (el 10 per a l'RFID i el 4 per a la SD) a estar apagats (**HIGH**) abans d'iniciar cap comunicació. Això garanteix que ambdós dispositius estiguin "adormits" i permet a l'ESP32 despertar-los d'un en un de forma segura només quan ho necessiti.

Inicialització Compartida:

- **SPI.begin(SCK_PIN, MISO_PIN, MOSI_PIN);**: Configura el bus SPI principal assignant explícitament els pins de maquinari de l'ESP32-S3.
- **SD.begin(SD_CS_PIN, SPI);**: És una línia vital. En lloc de cridar la inicialització estàndard, se li passa per referència l'objecte **SPI** global que acabem de crear. Així obliguem la targeta SD a fer servir els mateixos cables que l'RFID.

Gestió de l'Hora i Enregistrament (Datalogging):

- **configTime(gmtOffset_sec, daylightOffset_sec, ntpServer);**: Mitjançant la connexió Wi-Fi, l'ESP32 es connecta a un servidor de temps d'internet (**pool.ntp.org**) per establir l'hora actual i compensar el fus horari.
- Dins del **loop()**, un cop llegida i formatada en hexadecimal l'etiqueta RFID, s'executa **SD.open("/fichero.log", FILE_APPEND)**. El paràmetre **FILE_APPEND** indica al sistema d'arxius que no ha de sobre escriure el fitxer, sinó obrir-lo, anar fins a la darrera línia i afegir-hi la nova lectura (l'hora sincronitzada més l'identificador UID). Tot seguit s'executa **dataFile.close()** per consolidar l'escriptura.

Servidor Web Integrat: Paral·lelament a tot això, el **server.handleClient()** s'encarrega d'atendre peticions HTTP. La pàgina **handleRoot** construeix una interfície

senzilla que incorpora l'etiqueta HTML `<meta http-equiv='refresh' content='2'>`. D'aquesta manera, el navegador recarrega les dades en segon pla cada dos segons, mostrant l'identificador de l'última targeta llegida guardat a la variable `ultimaTargeta`.

Sortida pel Terminal de Control (i Fitxer)

En iniciar el sistema, el terminal sèrie detalla pas a pas l'èxit de les connexions (SPI compartit, SD, RFID, xarxa Wi-Fi i servidor de temps). Cada vegada que apropem una etiqueta diferent, el terminal de depuració ens avisa que la dada s'ha emmagatzemat correctament.

La sortida sèrie és la següent:

Plaintext

Lector RFID inicialitzat.

Targeta SD inicialitzada correctament.

Connectant a la xarxa: Nautilus

.....

Connexió establerta!

Adreça IP del servidor: http://192.168.1.50

Sincronitzant l'hora amb el servidor NTP...

Lectura detectada: A3 4F 8B 12

Registre guardat correctament a fichero.log

Lectura detectada: 55 B1 2C 09

Registre guardat correctament a fichero.log

Si accedim a l'adreça IP del servidor des de l'ordinador, veurem la interfície HTML indicant "Darrera lectura realitzada: 55 B1 2C 09". Paral·lelament, si extreiem la memòria física de l'ESP32 i la llegim, veurem com s'ha creat de forma autònoma el registre sol·licitat:

Contingut de `fichero.log`:

Plaintext

[11/05/2026 18:45:02] UID detectat: A3 4F 8B 12

[11/05/2026 18:48:15] UID detectat: 55 B1 2C 09

Anàlisi del Temps Lliure del Processador (IoT SPI Integrat: SD + RFID + Web)

L'anàlisi del rendiment d'aquest sistema final ens mostra el disseny més complex de la pràctica, on el microcontrolador ha de gestionar simultàniament la connectivitat de xarxa i el control de dos perifèrics esclaus pel bus SPI. El temps lliure de la CPU es veu afectat per la naturalesa de les tasques programades:

1. L'impacte de l'Espera Activa (Polling): Dins de la funció `loop()`, el processador no té un repòs constant, sinó que es troba en un estat de vigilància contínua.

- L'instrucció `server.handleClient()` interroga sense parar la pila de xarxa per veure si hi ha peticions web.
- Simultàniament, `mfr522.PICC_IsNewCardPresent()` satura el bus SPI amb ràfegues de control per detectar la presència magnètica d'una etiqueta. Aquesta configuració implica que, mentre no hi ha una targeta a prop, el processador dedica gairebé el 100% del seu temps a "preguntar" als sensors, deixant molt poc espai per a la Tasca Inactiva (Idle Task) del sistema operatiu.

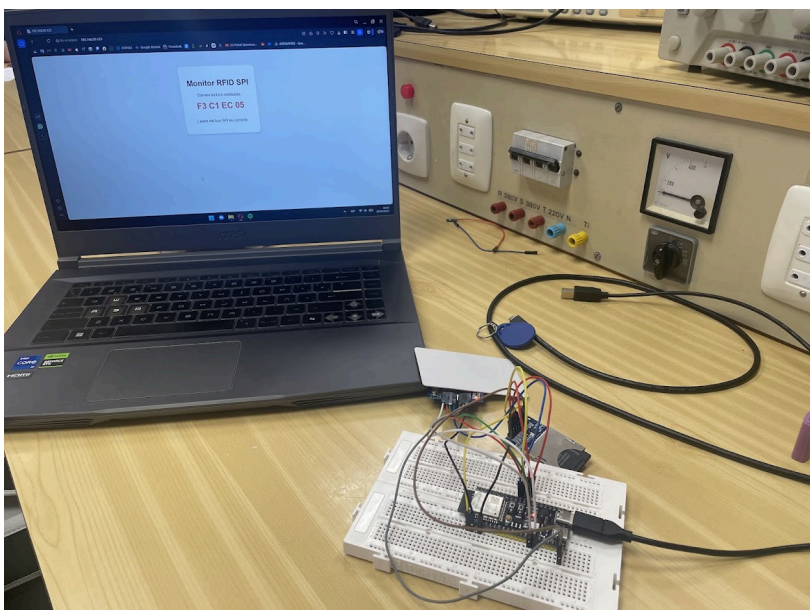
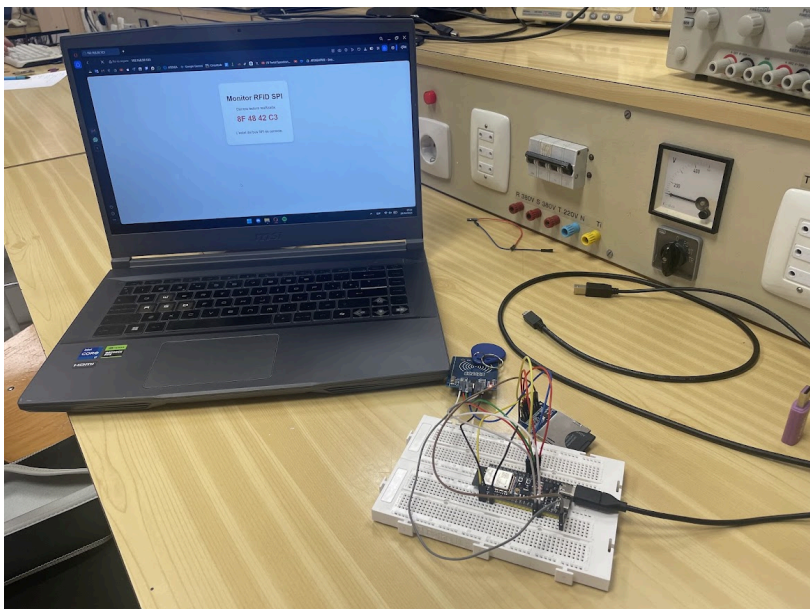
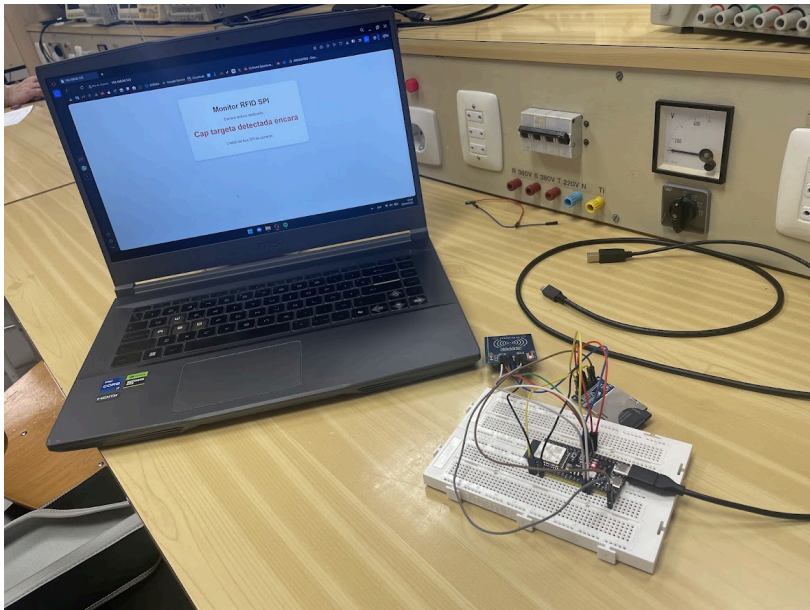
2. Latència d'Escriptura i Bloqueig de Maquinari: En el moment que es detecta una targeta, el perfil de càrrega canvia radicalment. L'operació d'obrir el fitxer `fichero.log`, escriure la cadena de text amb l'hora i tancar-lo és, amb diferència, la tasca més lenta des del punt de vista de la CPU.

- Durant aquests mil·lisegons d'escriptura a la targeta SD, el processador queda retingut esperant que la memòria física de l'esclau SPI confirmi la recepció de les dades. És un procés d'entrada/sortida (I/O) intensiu que bloqueja el flux principal del programa.

3. La Finestra de Repòs (`delay(500)`): Paradoxalment, el moment en què el processador gaudeix de més temps lliure real és just després de realitzar la feina més pesada. Després d'escriure a la SD i aturar la lectura de la targeta (`PICC_HaltA`), el codi executa un `delay(500)`.

- Sota el sistema operatiu FreeRTOS de l'ESP32, aquest mig segon no és un bloqueig buit, sinó que el *scheduler* suspèn la tasca principal i cedeix tot el control a la **Tasca Inactiva**. Això permet que el processador redueixi el seu consum i que les tasques de fons del sistema (com el manteniment de la connexió Wi-Fi) es gestionin amb total prioritat.

Conclusió: El sistema presenta un **temps lliure efectiu variable**. Mentre està en mode d'espera, la CPU treballa a ple rendiment per garantir una resposta immediata (alta responsivitat). No obstant això, gràcies a la programació de pauses controlades després de cada lectura, s'aconsegueixen finestres de repòs del 100% cada vegada que es registra un accés, equilibrant així la càrrega de treball i garantint l'estabilitat de les comunicacions de xarxa i el bus SPI compartit.



Conclusions

L'execució d'aquesta sisena pràctica ha representat l'assoliment d'una de les fites més complexes i útils en el disseny de sistemes encastats: el domini de les comunicacions d'alta velocitat mitjançant el bus **SPI** i la seva integració amb entorns de xarxa.

A partir dels resultats obtinguts i del codi desenvolupat, se n'extreuen les següents conclusions fonamentals:

- **Potència i Complexitat del Bus SPI:** S'ha pogut constatar que l'SPI és ideal per moure grans volums de dades (com llegir arxius d'una targeta SD) o garantir respostes gairebé instantànies (com la lectura d'una targeta RFID) gràcies a la seva naturalesa *Full Duplex*. No obstant això, aquesta velocitat comporta un repte de maquinari: la necessitat de gestionar acuradament les línies *Chip Select* (CS) per evitar col·lisions quan hi ha múltiples dispositius connectats en paral·lel.
- **El Compromís de l'Espera Activa (*Polling*):** A diferència d'altres pràctiques on la CPU podia descansar completament, la implementació del mòdul RFID ens ha ensenyat que el *polling* continu manté el processador a plena càrrega de treball. S'ha après la importància vital d'introduir instruccions d'aturada (**HalT**A) i petites pauses (**deLay**) per permetre que l'esclau finalitzi el seu procés i per cedir temps de còmput a les tasques de fons (Tasca Inactiva i Wi-Fi) de l'ESP32.
- **Sistema de Datalogging Funcional:** La capacitat de connectar-se a un servidor NTP d'internet per obtenir l'hora exacta i escriure-la automàticament de forma persistent (en mode afegir **FILE_APPEND**) dins d'un fitxer físic **.log** de la SD, suposa la creació d'un sistema de registre autèntic i professional, molt comú en les indústries d'automatització.
- **Assoliment del Paradigma IoT:** El darrer exercici integrador demostra el potencial de l'ESP32. S'ha aconseguit mantenir operatiu un servidor web asíncron altament responsiu mentre s'atenen, de forma completament transparent a l'usuari, les exigents lectures magnètiques i les lentes escriptures a memòria flaix.

En resum, aquesta pràctica no només consolida els coneixements sobre els protocols de comunicació sèrie síncrona, sinó que aporta l'experiència arquitectònica necessària per dissenyar i depurar sistemes IoT avançats on el maquinari local crític interactua constantment amb el món exterior.